

GRACE

Palak's editorial tracker: the design, reviewed and settled

Prepared by: Stephen Gadd · **Date:** 27 August 2026, revised 2 September 2026 · **For:** Ruth Mostern, Palak Vashist, Alexandra Straub · **Purpose:** response to "Tracking people, projects, and datasets — a working design for our editorial workflow"

Palak's draft describes a **coherent system**. The entity graph in the prose and the diagram agree with each other, the cardinalities are stated and correct, and the "how is it controlled / what values are allowed" discipline that Ruth asked for has produced exactly the data dictionary a build needs. This note sets out **four structural fixes**, answers the five questions Palak left open, and settles **one governance point** that had not previously been put on the table. All seven decisions are now made, and the build is under way.

Revised, 2 September 2026. Palak has now reviewed the working build on [staging](#). One of her seven points overturns a decision taken here: §8 argued for calling the pipeline record a *gazetteer*, and that was wrong. §8 has been rewritten and §13 records what the review changed.

The system is GRACE — *Gazetteer Register And Contact Engagement* — a Django app inside WHG, sitting alongside [datasets](#), [collection](#) and the rest. The name says what it holds: a register of the gazetteers, people, projects and sources we track, and a record of our dealings with them.

A note on terms. A contributor brings a **dataset**; once reconciled and published it is a **gazetteer**. One object, two life stages — the same thing we tell people on Map your Data. GRACE's pipeline record exists almost entirely during the first stage, so it is called a **Dataset**. That leaves "gazetteer" free to mean the two things it means everywhere else at WHG: a *printed* gazetteer, which is a **source** we might mine for places, and a *published WHG* gazetteer, which is a row in the Gazetteer Register. §8 sets out the reasoning, and why the first version of this note got it wrong.

Where this stands

#	Decision	Status	Outcome
1	Does GRACE point at the Gazetteer Register and read licence, rights, counts and coverage through it?	SETTLED	Yes — a nullable foreign key, with a local title so prospects can be named.
2	One Contact model linked to User, or two separate people-tables?	SETTLED	One, with an optional link and no duplicated fields.
3	Vocabularies as editable lookup tables, or frozen choices in code?	SETTLED	Tables by default — it is what preserves Palak's self-service.
4	Do we add an institution entity, or keep affiliation as free text?	SETTLED	Add it — permission to publish is granted by institutions.
5	Is the public form a suggest-a-source tool, or a general front door?	SETTLED	Suggest-a-source, with a visible untriaged intake state.

6	Personal data: lawful basis, consent, erasure path, encryption.	SETTLED	Legitimate interests, with the four obligations of §10 implemented in the build.
7	Terminology, and what the three registers are called.	REVISED	The pipeline record is a Dataset and the people register is People — see §8, rewritten after Palak's review. "Printed gazetteer" stays a source type. Registers: Catalogue · Engagement · Pipeline.

Nothing now blocks the build. It divides cleanly: the schema and the admin are a developer job, while the vocabularies, the views and the day-to-day shape of GRACE stay Palak's. Work is proceeding on the `staging` branch, where the whole team can review it before it reaches the live site.

1. What is working, and should not be touched

- **"Registers, not phases."** **KEEP** Work does not move in a line — a gazetteer can be in live correspondence with its owner *and* halfway through review. Naming these registers, in the old sense of a running official list, buys us out of a false sequence.
- **Two questions per field.** **KEEP** "How is it controlled" and "what values may it hold", answered for every field, is the difference between a spreadsheet and a system. The four-way classification maps cleanly onto Django: a pick-list becomes a lookup table, a link becomes a ForeignKey, a date is a date, and free text is confined to where consistency genuinely does not matter.
- **An engagement is with a person, not with a record.** **KEEP** The single best modelling decision in the draft. It is what makes "open one person and see every conversation" possible, and it is why the register survives a project being renamed or a gazetteer being withdrawn.
- **Action items as real tasks.** **KEEP** Palak's own correction — from a free-text plan to an owned, dated task — is right, and it is what makes the reminder rule work at all.
- **Print gazetteers folded into Sources.** **KEEP** Giving them their own record would duplicate the bibliography for no gain. One source type is simpler and correct.

2. Fix 1 — point at the Gazetteer Register rather than restating it

The proposed Dataset tracking status — *on our radar; in internal review; revisions requested; ready to publish; in external peer review; published; indexed; not indexed; on hold; declined* — conflates two axes. Some of those values are editorial judgements. Others are machine facts, and Palak flags the difficulty herself: "the dashboard stages are guesses... our technical director will know how they line up".

They line up onto something that already exists. The **Gazetteer Register** (`GazetteerRegistryEntry`) is WHG's canonical list of gazetteers, kept current by an idempotent push from the indexing side on every successful ingest. It is already the unified list this design wants: `entry_class` spans *authority* and *dataset*, so external authorities and WHG contributions sit in one table, and `gazetteer_type` already carries *standard / itinerary / network* — the same widening of "gazetteer" the terminology note describes.

So GRACE does not restate any of that. Its gazetteer record carries a nullable foreign key to the Register and reads through it:

```

class TrackedGazetteer(models.Model):
    # GRACE's own record. Exists from the moment something lands on our
    # radar – long before there is anything in WHG to point at.
    title = models.CharField(...) # what we called it when we first heard of it
    registry = models.ForeignKey('api.GazetteerRegistryEntry',
                                null=True, blank=True, ...)
    stage = models.ForeignKey('Stage', ...) # editorial values ONLY
    owner = models.ForeignKey(settings.AUTH_USER_MODEL, ...)

    @property
    def is_prospect(self): # no Register row yet – we are still chasing it
        return self.registry_id is None

```

What that buys is considerable, because the Register already holds — and the ingest push already maintains — the licence (a resolved link to the licence vocabulary), rights holder, CRediT-shaped contributors, structured citation text, source URL, record count, temporal extent, spatial coverage and an owner. None of it needs re-entering, and none of it can drift, because there is only ever one copy. Every machine fact therefore disappears from GRACE's own fields, and what remains is genuinely ours to decide: the editorial stage, the responsible person, review status, permission to publish, when it came onto our radar, and the notes.

Pointing at the Register rather than at `datasets.Dataset` also makes a whole class of work expressible that would otherwise fall outside the system. Much of our outreach concerns *authorities* — the Getty over TGN, ISAW over Pleiades — not contributed datasets. A Dataset link cannot represent those conversations; a Register link can.

Why GRACE still needs its own table

The natural next thought is to put the editorial fields straight onto the Register row. There is even a precedent for it: the inventory push deliberately omits the curatorial fields from what it writes, with a comment warning that adding them would "silently reset them on every push, blowing away staff curation". Protect-by- omission is a proven, documented pattern here.

It still should not be done, for three reasons — the first decisive:

- **The primary key rules it out.** A Register row is keyed by namespace — `gn` for an authority, `whg:1234` for a WHG dataset — and that identifier exists only once something has been ingested. A print gazetteer someone mentioned at a conference has no namespace, so it can have no row without minting a placeholder key that would then have to change when the thing became real, breaking every foreign key pointing at it. *Prospects are the majority of what GRACE tracks*, so this is the main case, not an edge case.
- **`status` is push-managed.** It is set on every push, with a single carve-out protecting *embargoed*. The editorial stage can therefore never be `status`; it has to be a field of GRACE's own regardless.
- **The Register row is public-facing.** The sources API, the Atlas gazetteer inventory and the Regions offcanvas all read it through a visibility gate. They use explicit field lists today, so nothing leaks by accident — but that discipline would have to hold across every future serializer once the same row also carried "permission unclear, chasing the archivist since March". That is a standing hazard for no benefit.

There is a lifecycle argument too: the Register row's existence is owned by the indexing side. A gazetteer we decline, or one later withdrawn, has no row — and its editorial history is exactly what we need to keep. Put plainly: **the Register is a gazetteer's published identity; GRACE is its editorial biography**, which begins earlier and outlives it.

Decision 1 — settled. GRACE's gazetteer record takes a nullable foreign key to the Gazetteer Register, keeping a local title so a prospect can be named before it has an identity. Every machine fact is read through that link, and the derived values (*published*, *indexed*, *not indexed*) come out of the editorial stage list. A row with no Register link *is* a prospect, so no vocabulary has to say so.

Over to Palak. Confirm the editorial stage list once the derived values are out. That half needs no technical input.

3. Fix 2 — much of the Catalogue is already built

Two more of Palak's tables substantially exist in WHG already, and re-creating them would leave us with two competing copies of the same facts.

People is mostly the existing user model

The user model already carries *name*, *given name*, *surname*, *affiliation*, *web page*, *ORCID*, *email*, *email confirmed*, *role*, *is active* and *preferred language* — and a *news permitted* flag, which is precisely the mailing-list consent flag the draft's *Mailing lists* field implies. Palak's *Identifier* is ORCID; her *active / dormant* is *active*.

The catch is that most people in the Catalogue will never have a WHG account — we have merely heard of them — and we must not create shell accounts for people we have only read about. So the right shape is a separate **Contact** model with an *optional* one-to-one link to the user record, and a firm rule that no field is stored in both places: where a Contact is linked, affiliation, ORCID and email are read through the link. This also turns Palak's own aside — "ultimately the people submitting will become the user" — from a note in the margin into a first-class relation.

Regions is already M49, and already built

Palak calls her region list a placeholder and asks for "a real scheme". WHG has one: the **regions** app is a complete UN M49 hierarchy — global / region / sub-region / intermediate / country — with parent and member relations, ISO alpha-2 and alpha-3 codes, Wikidata identifiers, multilingual labels, and actual geometry and hulls. The placeholder is dropped in favour of a many-to-many to Region, which also answers her question about multiple regions: yes, and it comes for free.

Decision 2 — settled. Build *Contact* with an optional link to the user model and no duplicated fields, rather than a People table from scratch. Use the existing Region model directly in place of the placeholder vocabulary.

4. Fix 3 — two entities are missing

Organisations

Affiliation is free text, and there is no institution record in the design — nor anywhere in WHG, so this genuinely is new. Permission to publish is granted by an archive, a library, a museum or a university, not by an individual, and *Permission to publish* is the field the entire licensing programme depends on. Projects are not institutions: a project ends, an institution does not, and the rights it granted outlive both the project and whoever we spoke to. A minimal Organisations model also gives us somewhere to record that we already hold a standing agreement with an institution before we approach a third researcher there.

Intake and triage

There is nowhere for a public submission to land. *Discovery source* offers *conference*; *referral*; *literature*; *flagged internally*; *other* and no value for a web submission, and nothing in the registers says "this arrived from the public and nobody has looked at it yet". The site's public suggestion form has to be built in Django anyway (§11), so the intake model and the form are designed together rather than one retrofitted to the other.

Related: *Sources* → *Associated datasets* is documented as "datasets it documents". That is a *documentation* relation. There is no *derivation* relation — the candidate gazetteer that was *extracted from* a given print source. Since turning print gazetteers into WHG gazetteers is the main reason the bibliography exists, losing that link would cost us the provenance chain for the whole programme.

Decisions 4 and 5 — settled. Add an Organisations model. Add *web form* to *Discovery source* and an unreviewed intake state. Split the Source → gazetteer link into "documents" and "derived from". The public form is a **suggest-a-source** tool — not a general front door that triages everyone and routes gazetteer owners on to the validate flow — and it shows its untriaged queue rather than hiding it.

5. Fix 4 — preserving Palak's self-service

A no-code database lets Palak add a field or a vocabulary term herself, in seconds, without a developer. Django ordinarily does not, and self-service is the reason the team wanted a no-code tool in the first place. If we build in Django and ignore that, we will have solved the data-modelling problem while recreating the bottleneck it was meant to remove.

The mitigation is a modelling choice, and it needs making at the start rather than retrofitting: **model each controlled vocabulary as its own table**, not as a `choices=` tuple. A choices list is frozen in code and needs a developer and a migration to change; a lookup table is editable in the Django admin in ten seconds. Reserve `choices` for the handful of lists that code actually branches on — where adding a value without changing logic would be a bug — and make everything else a table Palak owns.

Applied to the draft's vocabulary list, that means *people roles*, *mailing lists*, *project status*, *source type*, *priority*, *contact status*, *interaction channel*, *action-item status*, *review recommendation*, *outcome*, *discovery source*, *digitization status*, *content type* and *content status* are all tables. Only the editorial stage is a plausible candidate for `choices`, and only if notifications branch on specific values.

Decision 3 — settled. Vocabularies as tables by default, with an admin group giving Palak edit rights over them. The short list of code-critical vocabularies that stay frozen is to be agreed as they arise, and said so in the model docstrings so nobody later "tidies" them into `choices`.

6. Field-level corrections

Field	As drafted	Change to	Why
Gazetteer · licence, rights, record count, coverage, reconciliation	Fields on the tracker	Read through the Register link	§2 — maintained by the ingest push; not ours to type
People · Name, affiliation, ORCID, email	Free text, re-declared	Read through the optional user link	§3 — never store the same fact twice
People · Mailing lists	Pick-list incl. <i>none</i>	Reconcile with the existing consent flag	Consent already has a home; two flags will disagree

Region (projects & gazetteers)	Single pick-list, placeholder values	Many-to-many to Region	M49 with geometry, ISO codes and labels is already built
Gazetteer · Time period	Free text, e.g. <i>900-1200</i>	Keep the prose, add numeric start/end year	Unsortable and unfilterable as text — and a prospect has no Register row to read it from
Gazetteer · Language(s)	Free text	ISO 639-3, multiple	The data is multilingual; free text will not group
Discovery source	No web-form value	Add <i>web form</i>	\$4
Interactions · Logged by	Link to one of us	Drop	Duplicates the universal <i>added by</i> the draft says to assume on every record
Gazetteer · Status	Absent	Consider adding	People and Projects both carry one; the asymmetry is probably unintended

Three structural nits

- **There are four registers, not three.** Content is not engagement — it is output. The draft half-admits it ("content ideas: quick jots — planned pieces live in Content"). Either promote it, or drop the claim that each register is one working table; it is already untrue of the Catalogue, which is four.
- **Two *Responsible person* fields can disagree.** One sits on the Engagement, one on the tracked gazetteer, with no rule about which wins. The rule: one accountable owner per gazetteer, inherited by its engagements unless deliberately overridden.
- **The notification rule misses the failure mode that matters.** "Stage change notifies the responsible person" is a good, restrained rule — but what goes wrong in outreach is *stalling*, and a stall is an *absence* of a stage change. Nothing fires when an engagement has sat in *awaiting reply* for six weeks. The fix is nearly free: make *Next follow-up* mandatory whenever the stage is an open one, and the existing date reminder becomes a staleness alarm. Worth suppressing the notification when the responsible person is the one who made the change — otherwise the system spends its credibility telling people what they just did. All of this rides the existing mail machinery and scheduled tasks rather than needing anything new.

7. The five questions Palak left open

Question	Answer
1. Regions — placeholder list; can something sit in more than one?	Already solved, and yes. The Region model is a full UN M49 hierarchy with geometry, ISO codes, Wikidata ids and multilingual labels. Use a many-to-many to it and the multi-region question answers itself; single-region is the common case, not a constraint worth encoding.
2. Priority and stage — on the engagement, or on the catalog record?	The register logic answers it. A stage describing <i>the conversation</i> belongs on the Engagement; a stage describing <i>the thing</i> belongs on the thing. Contact status stays where it is (correct as drawn), and the editorial stage belongs to the tracked gazetteer. Priority is a property of the conversation: leave it on the Engagement.
3. Funding and grants — right home? Does it belong at all?	It belongs, for a reason the draft could not have known. Funder and grant number are part of the credit chain WHG already models — the Gazetteer Register carries CRediT contributors, rights holder and structured citation text. Keep <i>Funder</i> and <i>Grant number</i> on Projects, and note that they feed citation metadata. That is the justification, and it needs no new scope.

4. **Dashboard stages** — guesses, to be set with the technical director.

Half the question dissolves. The derived values are read through the Register link (§2). What remains is the genuinely editorial half, which is Palak's to define and needs no technical input at all.

5. **External peer review** — how much to build now?

None of it now — the draft is right to sketch and defer. Two notes for when we do: WHG's record-suggestion flow already runs *pending* / *approved* / *rejected* with a reviewer, timestamp and review note, and the Register's *embargoed* status already expresses "accepted but not yet public". Review comments could also reuse the existing comment model rather than inventing a parallel one.

8. On "dataset" and "gazetteer"

Rewritten, 2 September 2026. The first version of this section decided the opposite of what follows. It is replaced rather than amended, because the reasoning changed, not just the conclusion.

Palak's draft used "dataset" throughout, and the first version of this note treated that as the thing to correct. The argument ran: her table holds two kinds of thing — prospects nobody has asked about yet, and gazetteers WHG actually holds — and since "dataset" is wrong for the second, call the table *Gazetteers* and let the nullable Register link carry the distinction.

That was the wrong half to name it after. Reviewing the working build, Palak put the counter-argument plainly: a dataset is what a contributor submits and what moves through the pipeline toward publication, so the record on its way in should be called a Dataset.

Her stated premise — that a gazetteer is printed material, a reference work — is the bibliographic sense, and it is not how WHG uses the word anywhere else: the Gazetteer Register, Atlas ► Gazetteers, the twenty-two authority gazetteers behind </api/sources/>, the name of the platform. Adopting that premise would mean renaming half the site. But her conclusion holds on a different and stronger argument, and it is one WHG already makes to its users:

Decision 7 — revised. A contributor brings a **dataset**; once reconciled and published it is a **gazetteer**. One object, two life stages. GRACE's pipeline record exists almost entirely during the first, because at the moment it becomes a gazetteer the authoritative record is the Register row it reads through (§2). So the record is a **Dataset**, and the register of people is **People**.

The gain is not merely that one label is more accurate. It is that "gazetteer" inside GRACE now means exactly two things, and neither of them is the pipeline record: a **printed gazetteer**, which is a source type, and a **published WHG gazetteer**, which is a Register entry. The apologetic rider in the first version of this section — that *source type: printed gazetteer* was "the one place in the whole system where the word means a lead rather than a contribution" — is no longer needed, because there is no longer a competing sense to distinguish it from.

Two riders of its own. First, roughly half the tracked rows are **authority gazetteers** — TGN, Pleiades, CHGIS — which never arrive as a contributed dataset at all; we ingest them from a third party. Calling those "datasets" is a slight stretch. It is a smaller stretch than calling a contributor's spreadsheet a gazetteer, and the rule the team will actually use should be the one that fits the common case.

Second, the earlier claim that this was "a change to human-facing language, not a migration" no longer holds, and should not have been used to avoid the question. The Python classes were renamed too —

[TrackedGazetteer](#) → [TrackedDataset](#), [Contact](#) → [Person](#), [ContactRole](#) → [PersonRole](#), [ContactStatus](#) → [PersonStatus](#) — because a label that disagrees with the code underneath it is a trap for

the next developer. It is `TrackedDataset` rather than plain `Dataset` because `datasets.Dataset` already exists in this project and the two are deliberately *not* the same thing (§2). The rename was cheap: production held no GRACE tables, and Django renames models and columns without touching the rows.

GRACE keeps its expansion — *Gazetteer Register And Contact Engagement*. The acronym is the product's name, the Gazetteer Register is exactly what the Pipeline points at, and contact engagement is exactly what the Engagement register records. Only the table holding the people changed.

9. Naming the registers

Within GRACE, "Dashboard" has to go. WHG's dashboard is the logged-in contributor's own workspace — their gazetteers, their uploads — and this is a staff editorial queue with a different audience entirely. Both live in the same Django project, so the collision would be literal as well as conversational. Palak asks whether the two might merge; they should not, though her underlying instinct is sound and is answered by the user link in §3.

Settled. Catalogue · Engagement · Pipeline. "Pipeline" says what the third register is for — where a gazetteer sits on its way in — without borrowing a word that already means something else to our users.

10. Personal data

Holding this ourselves keeps the data at Pitt, under our own auth, backups and access control, with no third-party processor involved. But one detail makes the question pressing rather than routine.

WHG encrypts user emails at rest. The user model's email is an encrypted field, with an indexed one-way hash carrying equality lookups precisely because the column itself is unqueryable. A Contact table storing plain-text addresses for people who never signed up would hold *less*-protected personal data than we hold for people who did — a visible regression against the platform's own standard, in the same database. Whatever we build must meet the bar the user model already sets.

The rest follows from what the Catalogue is for. It will hold names, addresses, affiliations and mailing-list membership for researchers who never asked to be in a database. Because contacts and account-holders sit in one system, a subject access request spans both, which is an argument for settling retention and erasure while the models are still on paper.

The basis we are using

The lawful basis is **legitimate interests** (UK/EU GDPR Article 6(1)(f)), documented in a short Legitimate Interests Assessment — explicitly **not consent**.

Consent is the wrong instrument here and would actively hurt. It must be freely given, specific and informed *before* processing, which is circular when the whole point is recording someone we have not yet contacted. Worse, consent is withdrawable, and withdrawal would oblige us to erase the editorial record of a negotiation that actually happened — losing the provenance for rights we legitimately hold.

Legitimate interests fits the actual facts. The interest is scholarly and curatorial: identifying gazetteers and negotiating rights over them. The data is overwhelmingly *professional* — name, institutional affiliation, ORCID, work email — things a researcher who published a gazetteer would reasonably expect a gazetteer project to hold. The balancing test passes comfortably for that. It would **not** pass for home addresses, personal email, or anything special-category, so the model should make those hard to record rather than merely discouraged.

Four obligations follow, and they are the parts such projects usually miss. All four are being built in, not left as policy:

- **Article 14 transparency.** Because we collect from third parties rather than from the person, we owe a privacy notice within a month, or at first contact — whichever is sooner. The cheapest compliant route is a standing line in the first outreach email plus a section on the privacy page. This is a real obligation, not a nicety, and it is the one most likely to bite.
- **Keep the mailing list separate.** A newsletter is direct marketing, which needs consent under PECR/ePrivacy whatever our Article 6 basis for the contact record. That is exactly what the existing consent flag is — so legitimate interests covers the Catalogue entry, and consent covers the mail. One flag must not do both jobs, which is also why the draft's separate *Mailing lists* pick-list should go.
- **A real erasure path, not *dormant*.** Article 21 gives a right to object; *dormant* is a status, not erasure. Build erasure as **pseudonymisation, not cascade delete**: null the identifying fields on Contact and keep the Interaction rows with their dates and outcomes. The editorial history survives; the person does not. Design this in now — retrofitting it across Engagement, ActionItem and Interaction later is painful.
- **A retention rule. Review any contact with no interaction for three years.**

Plus the technical follow-on: match the user model, so an encrypted field with an indexed hash for lookups.

Decision 6 — settled. Legitimate interests is the lawful basis, and the four obligations above are implemented: the Article 14 notice at first contact, consent kept separate for the mailing list, erasure by pseudonymisation, and the three-year retention review.

Two riders, neither of which blocks the build. This is a controller's reasoned position rather than a legal ruling, and since Pitt hosts the platform their privacy office may want to see the Legitimate Interests Assessment. And it should be read alongside the withhold-by-default posture agreed for Indigenous data, which is stricter than GDPR and wins where the two differ.

11. The Baserow retreat, and what it left behind

An earlier attempt at this workflow ran on Baserow, an external no-code database, and left live code and content pointing at it. That route is now closed and the code-side cleanup is complete.

Loose end	Position
The licence sync a management command mirroring our licence vocabulary into Baserow	DONE Deleted. Its purpose was keeping an external copy in step, and the Gazetteer Register already resolves licences directly.
Settings and credentials	DONE The service-account settings are gone from the codebase and from the deployed environment files on both servers, and the WHG service account itself has been deleted.
The API token	PALAK The full read/write token that was circulated by email in the clear belongs to Palak's personal account , not to the service account, and Baserow scopes those per user. Only Palak can revoke it , under My settings → Database tokens.
The bibliography	DONE The whole workspace — 498 rows across four databases, including the 72-row Print Gazetteer Bibliography — has been exported and is held locally, off the public repository. See below.

The public form "Suggest a Source" in the Data menu	TO BUILD Still a redirect to an external Baserow form, so the site's public entry point for suggestions leads out of WHG entirely. It is rebuilt as a Django form writing to the intake model of §4, designed together with it.
The earlier Django prototype	READ A previous in-Django attempt — a lead model, an admin triage board, a public form — has been recovered from version history and written up. Its public form, spam handling and spreadsheet importer are being reused; its vocabularies and region fields are not.
The Baserow design note	DONE Marked superseded rather than deleted — it records a decision and its reversal, which is worth keeping.

A near miss worth recording. The earlier prototype's ~72 seeded bibliography rows were believed to be safe in our own database. They were not: that table had already been dropped from both the development and production databases, so the only surviving copies were the original spreadsheet and the Baserow table. The workspace has now been exported in full before anything further was touched.

Baserow itself has been left completely intact — nothing there was modified or deleted, and what becomes of that workspace is Palak's call.

The export contains names, email addresses, phone numbers and affiliations for people who never signed up to anything. It is deliberately held outside the public code repository, and must stay there until §10 is settled — version history cannot be erased on request, which is precisely the problem decision 6 exists to avoid.

12. What happens next

The build order, each step depending only on the ones above it:

1. **Vocabulary tables** — the lookup tables of §5, and the admin group that lets Palak edit them.
2. **Contact and Organisation** — with the optional link to the user model, and no duplicated fields.
3. **Gazetteers and Stage** — the nullable Register link of §2, plus the editorial stage list Palak defines.
4. **Engagement, ActionItem, Interaction** — with the one-owner rule and the staleness alarm of §6.
5. **The admin** — triage from the list view, as the earlier prototype showed is sufficient.
6. **The public suggest-a-source form** — replacing the outbound redirect, writing to the intake model.
7. **Import the exported content** — bibliography into Sources, prospects into Gazetteers, and the people tables under the basis settled in §10.

All seven steps are unblocked. The first import of real contact data is the point at which the Article 14 notice and the retention clock start applying, so both are in place before it runs.

All seven were completed and deployed to dev.whgazetteer.org in August 2026. §13 records what the team's review of that build changed.

13. What Palak's review of the build changed

Palak reviewed the working system on [staging](#) at the start of September 2026 and raised seven points. They are recorded here because two of them contradict decisions taken above, and because the reasoning matters more than the outcome.

#	Point	Outcome	What it changed
---	-------	---------	-----------------

1	Records should show what they are connected to, in both directions.	BUILT	The reason the tracker was wanted at all, and the build did not have it. Django's admin shows the relations a record points <i>at</i> and nothing pointing back, so a person's page said nothing about their conversations. Every change form now carries a read-only Connections panel covering both directions, with reverse counts on the lists. One real gap in the schema surfaced: there was no relation between a person and a source at all (author / compiler was free text), so "a contact to their sources" could not be shown. A link was added; the free-text string stays as the bibliography writes it.
2	People should be everyone, ourselves included, with status and mailing-list fields.	PART	Roles for our own side were added, and <code>PersonRole.is_internal</code> marks them — necessary, because Article 14 covers data obtained from someone <i>other than</i> the person, so it does not apply to colleagues, and without the exemption they would bury the notices genuinely owed. Active / dormant status already existed. Mailing-list membership was deliberately not built: the sending platform owns subscription state, because that is where the unsubscribe link points and where bounces are recorded, and a second copy would eventually contradict it. GRACE holds only whether the address still works, so a bounce marks the address dead without losing the person or their history. Growing the register to mailing-list size was also declined — everyone in it is owed an Article 14 notice, while subscribers got Article 13 at sign-up.
3	A dashboard the whole team can scan without training.	BUILT	The landing page now leads with a board of datasets on their way in — stage, permission, review, responsible, on-radar, and a link through to WHG — ordered furthest-along first, because everything starts "on our radar" and ascending order buried the live work under the backlog. Two requested columns needed care. Reconciliation is not a GRACE fact and is not stored: it is read through to <code>datasets.Dataset</code> , the same discipline as the licence read-through in §2. There is no first-indexed date anywhere to read — the Register has only <code>updated_at</code> and <code>reingest_finished_at</code> — so one has been asked for from the indexing side rather than invented here.
4	Missing dataset fields: language, record count, coordinates, licence, permission, format.	BUILT	Language and permission already existed and were merely buried. Record count, licence and rights existed as read-throughs — but only once linked, and a prospect, which is exactly when the question is live, has no Register row to read. Rather than break §2's rule against copying Register facts, the new <code>expected_*</code> fields hold a <i>different</i> fact: what someone told us during a negotiation. The Register wins the moment there is one; the expectation survives as a record of what was offered, and a mismatch is shown rather than hidden. Coordinates are a vocabulary, not a boolean, because "unknown" is the normal early state and is not "no".
5	A way to record reviews.	BUILT	Partly anticipated: the review-recommendation vocabulary was already seeded and the stage list already had "in internal review". A Review record now carries reviewer,

			type, sent, returned, recommendation, comments and shared-with-author. External peer review needs a vocabulary term, not a migration. The reviewer is a Person rather than a WHG account, which works only because the register is now everyone. A recommendation deliberately does <i>not</i> advance the stage on its own — implicit state changes are how a tracker stops being trusted.
6	Two missing vocabularies: reconciled-against, and regions.	BUILT	"Reconciled against" is <i>not</i> a hand-typed vocabulary: WHG already holds that list, and a typed copy would drift, so it is a link to the Gazetteer Register limited to authorities — the same names users see in Map your Data. Regions turned out to be already modelled on every model, exactly as §3 said. The reason nobody could find them is that <code>regions.Region</code> held zero rows, on dev and on production alike, so the picker was empty and the feature looked absent. Loaded on dev; production deliberately left alone, since populating it there would change Atlas behaviour.
7	Datasets and gazetteers are different things; and rename Contacts to People.	REVISED	Overturns Decision 7. See §8, rewritten.

Three defects outside GRACE surfaced while building this and were fixed: the custom user manager validated `given_name` and `surname` and then silently discarded them, so every account created through it displayed an ORCID-derived username where a name should be; the GRACE admin mirrored the default site's model admins during autodiscovery, so whether a licence autocomplete worked depended on `INSTALLED_APPS` order; and nothing in GRACE offered an account or a Register entry by name rather than by machine identifier.